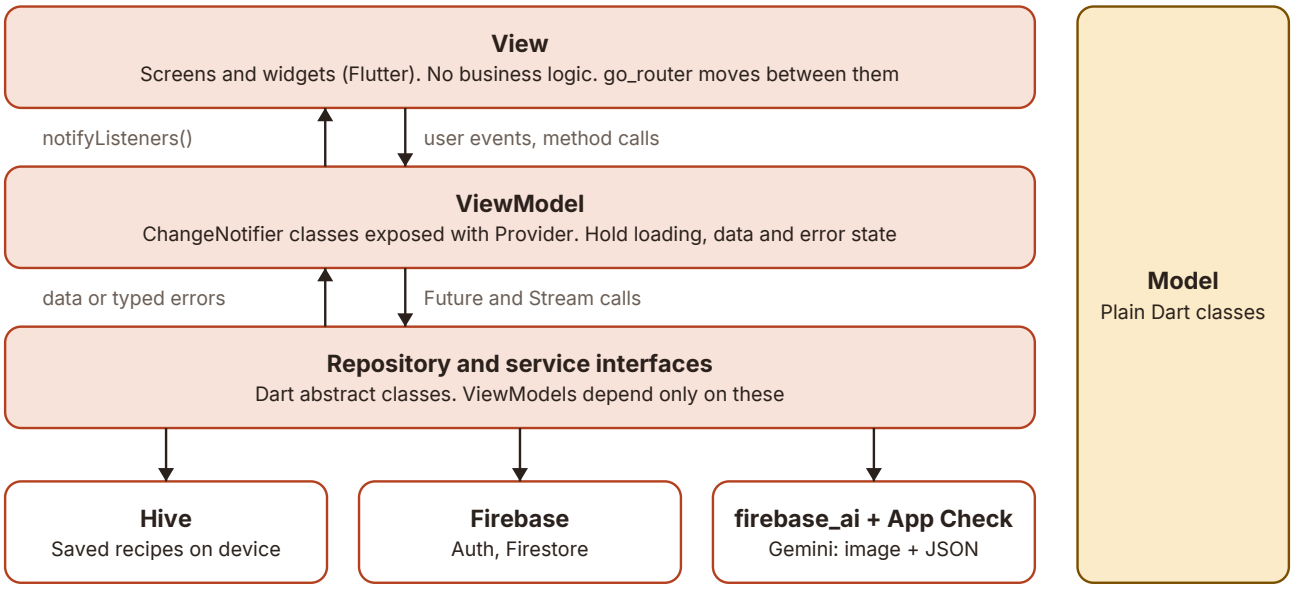
Architecture and Protocols

Dolaptan · MTM4681 Mobile Programming · Milestone · Team [TAKIM NO] · [AD SOYAD 1], [AD SOYAD 2]

1. Software design pattern: MVVM with Provider

Dolaptan uses **Model-View-ViewModel**, the clean structure the course asks for. Each ViewModel is a `ChangeNotifier` exposed with **Provider**, and widgets rebuild when it notifies. We chose Provider over Riverpod because it is simpler and enough for a two-person team on a short schedule.



Layer	Role in Dolaptan	Examples
Model	Plain Dart classes with JSON and Hive conversion	Recipe, Ingredient, Preferences, ListItem, Household, AppUser
View	Display state and forward events. They never call Firebase	HomeScreen, IngredientsScreen, RecipeListScreen
ViewModel	Screen state and rules, such as validating AI output and calculating match %	ScanViewModel, RecipeViewModel, ShoppingListViewModel
Repository / Service	Hide where data comes from, so ViewModels can be tested with fakes	RecipeAiService, SavedRecipeRepository, ShoppingListRepository

2. Screen map

Screen	go_router path	ViewModel	Main data
Login	/login	AuthViewModel	Firebase Auth
Home	/home	SavedViewModel	Recent saved recipes (Hive)
Scan	/scan	ScanViewModel	Camera image, AI detection
Ingredients	/ingredients	ScanViewModel	Editable detected list
Preferences	/preferences	RecipeViewModel	Time, servings, allergies
Recipe suggestions	/recipes	RecipeViewModel	3 AI recipes
Recipe detail	/recipes/:id	RecipeViewModel	Recipe, save and add-to-list actions
Saved recipes	/saved	SavedViewModel	Hive, works offline
Shopping list	/list	ShoppingListViewModel	Firestore real-time stream
Offline status	/offline	ConnectivityViewModel	Network state, saved recipes

A `redirect` in go_router sends signed-out users to /login. A `StatefulShellRoute` keeps the three tabs. Scan uses an `AnimationController` for the moving line. Screens open with iOS push transitions (`CupertinoPage`), Preferences opens as a sheet, and the recipe title moves into the detail screen with a `Hero`.

UI layer: Cupertino widgets on iOS and Android

The interface follows iOS design, so the View layer uses Flutter's Cupertino widgets. They now ship in the **cupertino_ui** package (version 1.1.5 on pub.dev, published by flutter.dev), which supports both Android and iOS. The same widgets render on both platforms, so one codebase gives the same look.

Design element	Flutter widget
App shell and routing	CupertinoApp.router with go_router, CupertinoPage for push transitions
Large-title navigation bar	CupertinoSliverNavigationBar
Tab bar	CupertinoTabBar inside a StatefulShellRoute
Inset grouped list	CupertinoListSection.insetGrouped, CupertinoListTile
Preferences sheet	showCupertinoSheet
Time and servings	CupertinoSlidingSegmentedControl
Allergy switches	CupertinoSwitch
Search, loading	CupertinoSearchTextField, CupertinoActivityIndicator
Swipe to delete	flutter_slidable (MIT, Android and iOS)
Icons	cupertino_icons (MIT)

Android: SafeArea keeps content clear of the Android status and navigation bars, and the system Back gesture pops the go_router stack. Inter is bundled as the app font, because Apple's SF Pro license does not allow use on non-Apple systems. The cupertino_ui page does not mention Liquid Glass, so bars use a plain translucent background. **[TO CHECK: Liquid Glass support in cupertino_ui before week 11.]**

3. Data model

```
Dart models
  Ingredient   name, isUncertain
  Preferences  minutes, servings, allergies[]
  Recipe       id, title, minutes, servings, usedIngredients[], missingIngredients[], steps[]
               photoPath (optional, empty until a real photo exists; the photo space is hidden)
               matchPercent = used / (used + missing), calculated in the app, never by the AI
  ListItem     id, name, addedBy, addedByName, checked, createdAt

Firestore (shared data only)
  users/{uid}                     name, householdId
  households/{id}                 name, inviteCode, memberIds[]
  households/{id}/items/{itemId}  name, addedBy, addedByName, checked, createdAt
  inviteCodes/{code}              householdId

Hive (device)
  saved_{uid}                     saved recipes, one box per user, for offline reading
  prefs                           last used preferences
```

Photos are compressed, sent only for analysis, and never stored in our database. Firestore security rules allow only household members to read or change that household's items. A user joins by reading one invite code document (no listing) and can add only their own uid to memberIds.

4. Key interfaces (Dart abstract classes)

Dart has no `protocol` keyword, so abstract classes define these contracts. ViewModels depend on them rather than on Firebase, so an implementation can be replaced by a fake in tests, or the AI provider can change without touching the UI.

```
abstract class AuthService {
  Stream<AppUser?> get userChanges;
  Future<void> signIn(String email, String password);
  Future<void> signUp(String email, String password, String name);
  Future<void> signOut();
}

abstract class RecipeAiService {
  Future<List<Ingredient>> detectIngredients(Uint8List photo);
  Future<List<Recipe>> suggestRecipes(List<Ingredient> items, Preferences prefs);
}

abstract class SavedRecipeRepository {
  Stream<List<Recipe>> watchAll();
  Future<void> save(Recipe recipe);
```

```
  Future<void> delete(String recipeId);
}

abstract class ShoppingListRepository {
  Stream<List<ListItem>> watchItems(String householdId);
  Future<void> add(String householdId, String name);
  Future<void> setChecked(String householdId, String itemId, bool checked);
}

abstract class HouseholdRepository {
  Future<Household> create();
  Future<Household> joinWithCode(String inviteCode);
}

abstract class ConnectivityService {
  Stream<bool> get isOnline;
}
```

5. Communication protocols

Protocol / technology	Purpose in the app
HTTPS (TLS)	Encrypts traffic between the app and Firebase services
Firebase Authentication	Signs users in with email and password and gives requests a verified identity
Firestore real-time listeners	Push shopping list changes to every household member as a Dart Stream
Firestore security rules	Check on the server who can read or change each document
Firebase AI Logic (firebase_ai)	Sends the photo and prompt to the Gemini Developer API through Firebase, so no Gemini API key ships in the app. Planned model: gemini-3.8-flash, kept in one constant so it can change
Firebase App Check	Proves that AI requests come from our real app. Uses Play Integrity on Android, and the debug provider during development
JSON with a response schema	Requests structured output (responseMimeType + responseSchema), then parses it into Dart models

6. Recipe generation sequence

(1) The View passes the photo to ScanViewModel. (2) RecipeAiService returns ingredients, and unclear ones are marked uncertain. (3) The user edits the list and sets time, servings and allergies. (4) The service requests 3 Turkish recipes as JSON in the format below. (5) The ViewModel validates the response, calculates match %, and exposes the cards. If a request or parsing fails, it retries once and then shows a retry message.

```
{
  "recipes": [{
    "title": "Patatesli Omlet",
    "minutes": 20,
    "servings": 2,
    "usedIngredients": ["patates", "yumurta", "soğan", "peynir"],
    "missingIngredients": ["tereyağı"],
    "steps": ["Patatesleri küp küp doğra.", "Tavada altın rengi olana kadar kavur."]
  }]
}
```